

LISTA, ISTA and FISTA: implementation and experiment

This extends the original `scripts/deblur.py` without modifying it. The agreed experiment is the primary coil, $z=3 \rightarrow 0.5 \mu\text{m}$, fitting all three component images. LISTA is supervised against the sharp COMSOL images and learns convolutional corrections to the exact padded physics operator, plus a threshold. The supplied notebook runs training on Kaggle; no full-data LISTA training result is claimed from local checks.

Theory and the exact variant

Gregor and LeCun's [original LISTA paper](#), equations 2–5 and Algorithm 3, unfolds shrinkage iterations into a network with learned input/recurrent maps and thresholds. Their supervision approximates sparse-optimization solutions. Our chosen sharp-field supervision is a different training target; this is a structured convolutional LISTA reconstruction variant, not a reproduction of that paper's experiments.

Let the original field units be μT and define

```
J(x) = 0.5 ||A x - b||2 + lambda ||x||1
A x = Crop(real(IFFT(H * FFT(Pad(x)))))
A* r = Crop(real(IFFT(conj(H) * FFT(Pad(r)))))
```

H is the unregularised joint kernel learned from $B_x/B_y/B_z$ at both heights, exactly as in `deblur.py`. Padding is 32 pixels by default. Keeping the intermediate crop is essential: A^*A is not generally one circular convolution with $|H|^2$ when padding and cropping are present.

The step is $\alpha = 0.99 / \max(\text{abs}(H)^2)$. The maximum is an upper bound for $\|A\|^2$; crop and zero-padding cannot increase the norm. With the half-squared data term, the gradient is $A^*(Ax-b)$; there is no factor of two.

ISTA:

```
x_next = soft(x - alpha*A*(A*x-b), alpha*lambda)
```

FISTA:

```
x_next = soft(y - alpha*A*(A*y-b), alpha*lambda)
t_next = (1 + sqrt(1 + 4*t2))/2
y_next = x_next + ((t-1)/t_next)*(x_next-x)
```

FISTA starts with $y=x_0$, $t=1$ and returns x , not its extrapolated y . This is the constant-step method in Beck and Teboulle's [FISTA paper](#), §4. Its objective-gap rate is $O(1/k^2)$, versus $O(1/k)$ for ISTA under the stated convex assumptions. This is not a guarantee of lower reconstruction error at every iteration, and standard FISTA need not have a monotone objective.

Our learned maps are:

```
W_b = alpha*A* + C_b
W_x = I - alpha*A*A + C_x
x_next = soft(W_b*b + W_x*x, theta)
```

C_b and C_x are bias-free, single-input/single-output, zero-padded 9×9 convolutional linear maps. The weights are tied across the 8 default layers, as is a scalar nonnegative threshold. There are 163 trainable parameters with these defaults (2×81 weights + 1 threshold). The fixed H is a buffer, not a trainable parameter. PyTorch's Conv2d convention is cross-correlation; learned coefficients need no analytic kernel reversal.

Corrections start at zero and $\theta = \alpha \cdot \lambda$, so before training every layer equals ISTA, including boundaries. The corrections' small spatial support does **not** truncate the fixed physical operator's long-range response. After learning, the effective maps need not be adjoints, positive, contractive or minimizers of J . There is no LISTA convergence guarantee and running more layers than trained is not supported. Only the final layer is supervised; intermediate layers are diagnostic outputs.

All methods default to $x_0 = b$ to match existing repository ISTA. `--initial zero` is available and applies consistently to all three; original LISTA usually starts from zero. This deviation is explicit.

Units and loss

All six field arrays are loaded using the existing float32-T-to- μ T scaling. Kernel fitting uses NumPy float64. Training/inference use PyTorch float32 without AMP or TF32. Define a single scale $q = \max(\text{abs}(\text{all sharp and blurred fields}))$, then use $u = x/q$, $v = b/q$ and $\lambda_{\text{normalized}} = \lambda_{\text{uT}}/q$. Thus

$$J(q \cdot u)/q^2 = 0.5 \cdot \|A \cdot u - v\|^2 + (\lambda_{\text{uT}}/q) \cdot \|u\|_1.$$

Scaling both field arrays but keeping λ unchanged would change the optimization problem. Diagnostics restore physical units: objectives multiply by q^2 , fields/RMSE/proximal-gradient residuals by q . The tests check this scaling against the original NumPy operators.

Training minimises mean squared error $\text{mean}((\text{LISTA}(v) - \text{sharp}/q)^2)$ over all pixels and all three component images. It is not the L1 objective J . Adam's learning rate defaults to $1e-4$; gradient norm is clipped at 1; the threshold is projected to nonnegative values after each update. Threshold parametrisation avoids freezing an extremely small initial value behind a softplus derivative. The reported best checkpoint is selected by **post-update training MSE**, not validation or test error.

Defaults of 8 layers, 9×9 filters, 200 epochs and learning rate $1e-4$ are explicit starting configurations, not theoretically optimal hyperparameters. They are editable in the notebook. The physical $\lambda = 1e-4$ and $\text{pad} = 32$ match the original ISTA defaults.

Data and comparison protocol

There are three whole-image training examples: B_x , B_y and B_z from the same coil. Each is processed as one channel, with one shared model. There is no random patch split, artificial resampling, geometric

augmentation or independent test set. Training preserves complete images to avoid silently changing the forward operator at patch boundaries.

The reference and target images are used to fit H for all methods, and the reference images additionally supervise LISTA. This is the agreed **same-sample fitting experiment**. Any inference about another height, coil or unseen measurement requires a separate data protocol and training experiment.

Two T4s are used through DataParallel to split the three-image training batch 2+1. MSE is computed after gathering predictions, so both chunks receive correct sample weighting. Both GPUs do not automatically imply twice the speed with so few images. All timing comparisons run on the **same single GPU**, with the same float32 dtype and three-image batch. See the official [DataParallel documentation](#).

Outputs compare:

- ISTA/FISTA at the same number of updates as LISTA's trained depth.
- ISTA/FISTA at the longer baseline budget (200 updates by default).
- L1 objective and proximal-gradient RMS, measuring optimization progress.
- Physical RMSE, range-normalised RMSE and relative L2 error, measuring reconstruction agreement.
- Warmed, synchronized median inference latency, separately from kernel fitting and training time.

A LISTA layer performs the physics gradient **plus** learned convolutions; it costs more than an ISTA iteration. Equal depth alone is not equal computation. No general speedup is claimed before measuring the Kaggle run. Training time includes post-update evaluation and best-weight copying; inference latency excludes diagnostics and disk writes. CUDA timing explicitly synchronizes before and after each measured call.

The proximal-gradient mapping is $(x - \text{soft}(x - \alpha * \text{grad } J_{\text{data}}, \alpha * \lambda)) / \alpha$. Its RMS is zero at an L1 optimum and is reported in physical units. It does not require a ground-truth solution or invent a reference optimum. For LISTA it is only a diagnostic because LISTA optimizes supervised MSE.

Files and use

- `scripts/fista.py`: standalone NumPy FISTA, with saved reconstruction, kernel and JSON metadata.
- `scripts/iterative.py`: shared comparable NumPy ISTA/FISTA and objective/stationarity diagnostics.
- `scripts/torch_inverse.py`: batched differentiable padded operator and GPU ISTA/FISTA.
- `scripts/lista.py`: `ConvLISTA`, checkpoint loading, and standalone training CLI.
- `scripts/compare_lista.py`: whole-image supervised training, comparison and provenance.
- `scripts/plot_comparison.py`: rebuild plots from saved arrays/JSON.
- `tests/test_iterative.py`, `tests/test_lista.py`: numerical and learning checks.
- `notebooks/ISTA_LISTA_FISTA_Kaggle.ipynb`: upload this notebook to Kaggle.
- `kaggle_lista_bundle.zip`: add this generated bundle as a Kaggle dataset. It contains code, tests, metadata and only six necessary `.numpy` arrays; no raw CSV upload is required.
- `scripts/prepare_kaggle.py`: rebuild the notebook and bundle after code changes.

The portable bundle retains original `grid.json` for provenance, although it only includes reference .5 and target 3 components. Other heights and Bnorm listed in that original metadata are deliberately not

packaged.

Example local commands, using a suitable PyTorch environment:

```
python -m unittest discover -s tests -v
python scripts/fista.py --z 3 --comp Bz --n-iter 100 --output runs/fista_z3
python scripts/lista.py --data-root . --z 3 --device cuda --gpus 2 \
  --layers 8 --kernel-size 9 --epochs 200 --lr 1e-4 \
  --baseline-iterations 200 --pad 32 --lambda-ista 1e-4 \
  --checkpoint-layers --output runs/compare_z3
```

Existing output directories are refused to prevent overwriting prior runs. `--checkpoint-layers` trades recomputation for lower training memory without changing the forward calculation. It is enabled in the notebook. Full-image training remains GPU work; the notebook leaves configuration and the training cell visible for you to run.

Kaggle's installed PyTorch/CUDA pairing is used, not replaced with a CPU wheel. NumPy, pandas and Matplotlib are also required. The notebook logs all actual versions/device names; the reproducibility controls disable TF32/AMP and request deterministic algorithms. As PyTorch's [reproducibility notes](#) explain, identical seeds do not guarantee bitwise equality across different versions/devices.

Saved artifacts

Each comparison directory contains:

File	Contents
<code>lista_best_training.pt</code>	CPU state dict, physics buffers, exact original norm bound/step configuration, scaling, architecture, selected epoch and loss.
<code>kernel.npy</code>	The NumPy complex128 empirical transfer before conversion to float32 buffers.
<code>reconstructions.npz</code>	Sharp reference, observation, original untrained LISTA, trained LISTA, ISTA and FISTA arrays, all in μT .
<code>metrics.json</code>	Per-iteration objectives/errors, training history, same-depth/long-run scores and latency samples, parameter changes, configuration, environment and SHA-256 input/source hashes.
<code>reconstructions.png</code>	Full-image reference/observation/three-method comparison; common percentile display scale per component.
<code>convergence.png</code>	Objective, reference NRMSE and stationarity versus iteration/layer.
<code>training_and_latency.png</code>	Training loss and inference latencies.

To reload a checkpoint without fitting H or retraining:

```
import torch
from lista import load_checkpoint
model, metadata = load_checkpoint('run/lista_best_training.pt',
device='cuda:0')
# observed_uT: torch.float32 [batch,1,1400,1400], on cuda:0
with torch.no_grad():
    reconstructed_uT = model(observed_uT / metadata['scale_uT']) *
metadata['scale_uT']
```

The loader restores the original double-fit norm bound rather than recomputing it from rounded float32 buffers. This preserves the exact forward step of the saved model. Checkpoint loads use `weights_only=True`.

Verification and limitations

Local checks pass for: padded adjoint against an explicit matrix transpose; FFT operator/autograd against NumPy; ISTA equivalence to existing code; FISTA against an explicit-matrix recurrence and an exact L1 solution; all-layer LISTA initialization equivalence; learned weight/threshold gradients against finite differences; physical-unit normalization; checkpointed versus ordinary gradients; training loss/weight updates; and checkpoint round-trip output equality.

The two-CUDA-device gradient-weighting check is included and runs in the Kaggle notebook. It is skipped locally because CUDA is unavailable. A tiny synthetic CLI run checks artifact creation; a two-iteration full-resolution FISTA check checks the real-data entry point. These are verification runs, not scientific performance results. Full coil LISTA training, its accuracy and any GPU speedup remain to be measured by the supplied notebook.